RIME–DBO-Based Capacity-Feasible Task Offloading and Resource Allocation in Mobile Edge Computing

Haoru Su*, Jie Bai, and Jiangyi Zhou
College of Computer Science, Beijing University of Technology, Beijing 100124, China
*Corresponding author: Haoru Su (suhaoru@bjut.edu.cn).
E-mail addresses: suhaoru@bjut.edu.cn (H. Su), baijie0219@gmail.com (J. Bai), zhoujiangyi@bjut.edu.cn (J. Zhou).

Abstract

Mobile edge computing (MEC) extends the computational capabilities of resource-constrained devices by leveraging nearby edge servers and remote cloud resources. However, heterogeneous task offloading requires the joint optimisation of discrete execution-node selection and continuous computing-resource allocation under limited node capacities. To address this challenge, this paper proposes a capacity-feasible task-offloading and resource-allocation framework based on a hybrid RIME–dung beetle optimisation algorithm (RDHO). The framework incorporates feasible execution-node encoding, CPU-cycle-rate decoding, deterministic capacity repair, population-based optimisation, incumbent-solution tracking, and optional local refinement. Comparative experiments show that the complete RDHO framework achieves competitive overall performance in the end-to-end evaluation. Nevertheless, under strictly controlled conditions with common initialisation, unified decoding and repair mechanisms, and equivalent computational budgets, dung beetle optimisation exhibits better population-stage performance. Its advantage remains statistically significant after applying the same refinement procedure, although the magnitude of the difference varies with the penalty configuration. These findings support the effectiveness of the integrated capacity-feasible framework in the evaluated synthetic scenarios, while not supporting universal superiority of the hybrid population-update mechanism.

Keywords: mobile edge computing; task offloading; computing resource allocation; capacity feasibility; controlled evaluation.

1 Introduction

Latency-sensitive sensing, Internet of Things (IoT), fifth-generation (5G) and emerging sixth-generation services generate heterogeneous workloads close to users. Mobile edge computing (MEC) reduces service distance by supplementing devices with edge and cloud resources [1,2].

A practical offloading decision must select a legal execution node and allocate finite processor capacity. The resulting categorical-continuous decisions interact through transmission paths, per-node CPU budgets, device-side energy and task-specific service thresholds.

Existing studies primarily optimise energy and latency, while freshness-aware work adds Age of Information (AoI) and user-oriented work considers Quality of Experience (QoE) or fairness [3-7]. These criteria are mathematically coupled rather than independent in the proposed formulation: in the present model AoI contains service delay and QoE transforms delay, energy and freshness into bounded utility.

To address this problem, this study develops an RDHO-based optimisation framework that adapts RIME- and DBO-inspired population movements to legal-node selection, allocated CPU-cycle-rate decisions and deterministic capacity repair. The framework treats the benefit of the hybrid population mechanism as an empirical question rather than an assumption.

The contributions are threefold. First, the paper formulates a capacity-feasible three-tier MEC model with legal local, edge and cloud execution, allocated CPU-cycle-rate decisions, aggregate node capacity, device-side energy, delay, a periodic AoI approximation over the decision epoch, priority-weighted aggregate QoE and priority-neutral active-user fairness. Second, it develops an RDHO-based integrated framework with legal-node population encoding, allocated-rate decoding, deterministic reassignment and projection, RIME-DBO-inspired updates, dynamic search guidance, incumbent

tracking under a fixed reporting objective and optional refinement. Third, it reports paired controls with equal numbers of function evaluations (NFE), common initialisation/repair/refinement, Wilcoxon tests, Holm correction, rank-biserial effects and wins/ties/losses.

Under the configured end-to-end procedures, RDHO-full obtains mean reporting fitness 0.947. This configured comparison has unequal NFE and evaluates complete solvers, not an isolated RIME-DBO population operator. The strictly controlled results reported in Section 6 show the narrower population-stage and common-pipeline conclusions.

Section 2 reviews related work. Sections 3 and 4 define the physical system and P1. Section 5 presents the RDHO-based capacity-feasible optimisation framework. Section 6 separates configured end-to-end and controlled population-update evidence, and Section 7 concludes.

2 Related Work

Following MEC surveys, related work is organised by solution methodology: model-based optimisation, learning-based methods and metaheuristic search [8-10]. Objective dimensions are discussed within these classes.

Model-based studies jointly allocate communication and computation resources or coordinate edge-cloud execution under explicit system constraints [3,11,13,14]. Legal collaboration topology and resource budgets must be represented before optimising task placement.

Learning-based methods adapt policies to changing states and have been applied to online offloading, vehicular cooperation and blockchain-enabled edge-cloud systems [12,15-17]. Their training and data requirements motivate complementary transparent offline optimisation for a fixed decision epoch.

Freshness-aware MEC incorporates AoI alongside service time [18-20]. The earlier TLBO-HHO method is included as an AoI-aware metaheuristic baseline [21]. QoE and fairness studies represent service acceptability and user balance [5-7,22-26].

Metaheuristic methods search nonlinear mixed spaces without a learned policy. Relevant mechanisms include HHO, GA, TLBO, RIME, DBO and enhanced sparrow search [27-36]. A hybrid label alone does not establish benefit; initialisation, evaluation budget, population updates and refinement require controlled comparisons.

Representative studies on parking-edge cooperation, blockchain-enabled edge-cloud offloading, cooperative deep reinforcement learning and potential-game strategies show how collaboration topology, resource coupling and service requirements alter offloading decisions [37-40]. A remaining methodological gap is a reproducible framework that combines capacity-feasible allocated CPU-cycle-rate decisions with priority-weighted system QoE and priority-neutral active-user fairness, while experimentally separating initialisation, evaluation budget, population updates and refinement.

The present work addresses that combined methodological and evaluation gap for a simulated cloud-edge-device epoch; it does not claim that allocated CPU-cycle-rate modelling alone is novel, nor does it claim communication-resource optimisation, online queue scheduling or deployment validation.

3 System Model

3.1 MEC Network, Assumptions, and Task Model

We consider a three-tier MEC system and assume that all tasks are evaluated within one decision epoch. Let D, $\mathscr{E}$, C, and T denote the sets of source devices, edge servers, cloud servers, and tasks, respectively. The unified execution-node set is $N = D \cup \mathscr{E} \cup C$, and $N_i \subseteq N$ denotes the legal execution-node set of task $i \in T$. Globally unique identifiers are used for all nodes, and $M = |N|$. We further assume that the network topology and communication rates remain fixed throughout the epoch. This abstraction excludes association and rate adaptation from P1 and allows the optimisation to focus on execution-node selection and allocated CPU-cycle rates. Figure 1 illustrates the legal execution choices and the shared aggregate CPU-cycle-capacity pools.

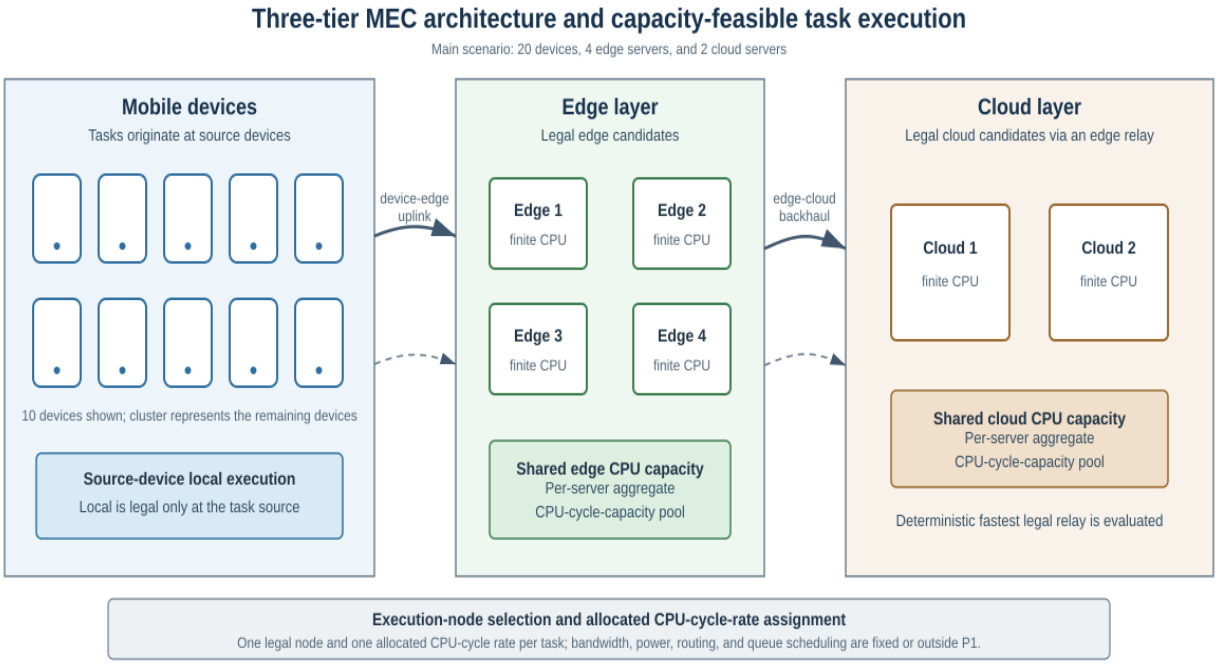

Fig. 1. Three-tier cloud-edge-device architecture with legal execution-node choices and shared aggregate CPU-cycle-capacity pools.

We assume that each task $i \in T$ is generated by one source device $d(i) \in D$. Local execution is legal only at $d(i)$. Because communication rates are fixed, an edge server $e \in \mathcal{E}$ belongs to N_i when the device-edge rate $R^{de}_{d(i),e}$ is positive, whereas a cloud server $c \in C$ belongs to N_i when at least one reachable edge server provides a positive edge-cloud rate $R^{ec}_{e,c}$. For cloud execution, we assume deterministic relay selection: $e_i^{\star}$ denotes the legal edge server that minimises the task-specific input-transmission delay. Consequently, routing and bandwidth allocation are outside the decision space. Task i is characterised by the tuple in (1).

$$\tau_i = \left(b_i, c_i, \Delta_i, T_i^{max}, E_i^{bud}, A_i^{max}, \beta_i, \pi_i, d(i)\right) \tag{1}$$

In (1), b_i is the input-data size, c_i is the required number of CPU cycles, Δ_i is the update-generation interval, T_i^{max} is the maximum tolerable delay, E_i^{bud} is the device-side energy budget, A_i^{max} is the AoI threshold, β_i is the battery ratio, and π_i is the task-priority weight. All tuple components are exogenous task attributes and are assumed fixed during the decision epoch.

We assume that tasks are mutually independent, with no precedence constraints or inter-task data dependencies. Each task is indivisible and is assigned to exactly one legal execution node within the decision epoch; task migration and pre-emption during execution are not considered.

Binary $x_{ij} \in \{0,1\}$ indicates whether task i is assigned to node $j \in N_i$, and each task satisfies $\sum_{j \in N_i} x_{ij} = 1$. The selected execution layer $z_i \in \{local, edge, cloud\}$ is derived from the chosen node. The continuous variable f_i is the allocated CPU-cycle rate of task i (cycles/s), and F_j is the aggregate CPU-cycle capacity of node j. Feasibility requires $\sum_{i \in T} x_{ij} f_i \le F_j$ for every node $j \in N$. For local dynamic voltage and frequency scaling (DVFS), f_i is an effective execution frequency; for edge or cloud execution, it denotes the task-specific share of the selected server's aggregate CPU-cycle capacity, measured in cycles/s; it is neither the server's physical clock frequency nor a task-completion throughput.

Each search individual $Y \in [0,1]^{N_T \times 2}$ contains one row $y_i = (u_i, r_i)$ per task. The node coordinate u_i indexes the sorted legal-node set N_i, whereas the resource coordinate r_i is decoded to a tentative allocated CPU-cycle rate. Both coordinates are clipped to $[0,1]$ before decoding. In particular, u_i and r_i

are algorithmic coordinates rather than physical resource quantities. After decoding and Algorithm 1 repair, the internal individual yields the physical solution used in P1.

3.2 Delay, Energy, and AoI Models

For node j with minimum allocatable CPU-cycle rate f_j^{min} and aggregate capacity F_j, the tentative request is $f_i^{req}=f_j^{min}+r_i(F_j-f_j^{min})$.

$$f_i^{req}=f_j^{min}+r_i\left(F_j-f_j^{min}\right),0\le r_i\le 1,x_{ij}=1 \tag{2}$$

For a decoded assignment, T_j denotes the set of tasks assigned to node j, n_j is the corresponding task count, and R_j is the residual CPU-cycle capacity after all assigned tasks receive the node-specific minimum rate. These quantities are defined separately in (3a)–(3c).

$$T_j=\left\{i\in T\,/\,x_{ij}=1\right\}. \tag{3a}$$

$$n_j=\left|T_j\right|. \tag{3b}$$

$$R_j=F_j-n_jf_j^{min}. \tag{3c}$$

Algorithm 1 uses (3a)–(3c) to perform deterministic legal-node reassignment before excess-capacity projection. At each pass, it computes minimum-rate usage, visits overloaded nodes in ascending global-ID order, and visits the tasks assigned to each overloaded node in descending task-index order. For each task, it considers only legal alternative nodes that can accept the task at their minimum allocatable rates. Among these alternatives, Algorithm 1 selects the node with the largest post-move residual minimum-rate slack, breaking ties by the smallest global node ID. Usage is recomputed in the next pass, and at most N_TM passes are allowed.

If this deterministic sequence cannot construct a feasible minimum-rate reassignment for a decoded candidate, evaluation raises ValueError. This outcome does not prove that the complete scenario is infeasible under every possible reassignment sequence. The exception propagates and aborts the affected optimiser run; the implementation does not assign positive infinity, retain the parent, or resample. The controlled evaluator records each failure before re-raising, and all reported controlled runs recorded zero repair failures. Because every target is checked for minimum-rate feasibility before a move, a completed repair does not introduce an overload cycle.

$$\begin{aligned} q_i&=max\left\{f_i^{req}-f_j^{min},0\right\},i\in T_j,\\ f_i&=f_j^{min}+\begin{cases} q_i, & \sum_{k\in T_j}q_k\le R_j,\\ R_jq_i/\sum_{k\in T_j}q_k, & \sum_{k\in T_j}q_k>R_j. \end{cases} \end{aligned} \tag{4}$$

Algorithm 1. Deterministic legal-node reassignment and excess-capacity projection shared by all solvers.

Require:	Decoded legal node IDs and requested allocated CPU-cycle rates; node minima and aggregate capacities.
Ensure:	Legal unique assignments, allocated rates within selected-node bounds and aggregate capacity; repair targets are pre-feasible, so no overload cycle is introduced.
1	For at most N times number-of-nodes passes, compute minimum-rate use of each node.
2	Visit overloaded nodes in ascending global ID; visit their currently assigned task indices in descending order.
3	For a task, consider legal nodes other than its current node; retain only targets that remain within capacity after receiving that task at the target minimum rate.
4	Move the task to the feasible target with largest post-move residual slack; break ties by smallest global node ID. Recompute use on the next pass.

5	If the deterministic order cannot construct a feasible minimum-rate reassignment for this decoded candidate, raise ValueError and abort the affected optimiser run; do not infer global scenario infeasibility, assign +infinity, retain the parent or resample.
6	When minima fit, set every task to its node minimum. Retain requested excess if total excess fits residual capacity; otherwise proportionally scale only the excess.

Once every minimum allocation fits, each node retains decoded requests if their total excess over task minima fits its residual capacity; otherwise only that excess is proportionally projected. Thus every allocated CPU-cycle rate remains no lower than the selected node minimum, and no non-overloaded request is silently saturated.

$$T_i=\begin{cases} c_i/f_i, & z_i=local, \\ b_i/R_{d(i),s_i}+c_i/f_i+\delta_E, & z_i=edge, \\ b_i/R_{d(i),e_i^*}+b_i/R_{e_i^*,s_i}+c_i/f_i+\delta_C, & z_i=cloud, \end{cases} \tag{5}$$

No heuristic congestion attenuation or M/M/1 queue is added, because finite CPU capacity and repaired allocated CPU-cycle rates represent processor scarcity. Fixed edge/cloud overheads are synthetic simulation parameters.

For local execution, device-side dynamic-voltage-and-frequency-scaling energy is

$$E_i=\kappa_{d(i)}c_i f_i^2, z_i=local \tag{6}$$

For edge or cloud offloading, the selected edge or legal cloud relay is fixed by the decoded path. The reported device-side energy includes uplink transmission only; infrastructure computation and backhaul energy are outside the boundary and this metric is never labelled total system energy.

$$E_i=P_{d(i)}\frac{b_i}{R_{d(i),e_i^*}}, z_i\in\{edge, cloud\} \tag{7}$$

The task-result size is assumed negligible relative to the input size; result-return delay and downlink energy are omitted. Device-edge rates are independently sampled U(8,30) Mbit/s then independently removed with probability 0.10; edge-cloud rates are independently sampled U(60,150) Mbit/s then independently removed with probability 0.05. If a device has no positive edge link, one uniformly selected edge is resampled from U(8,30); if a cloud has no incoming positive backhaul link, one uniformly selected edge is resampled from U(60,150). Repair changes only the absent link and does not alter existing sampled rates.

We assume periodic update generation and no explicit queue backlog. Rather than simulating successive arrivals, replacement, or queue accumulation, the model evaluates a delay-shifted sawtooth over one update period within the current decision epoch, giving the average approximation

$$A_i=\frac{\Delta_i}{2}+T_i \tag{8}$$

AoI is consequently delay-coupled, not independent. Equation (8) is a model-based freshness approximation for the fixed-epoch setting, not a steady-state queue-aware or peak-AoI formulation; no claim is made about backlog dynamics when the realised delay exceeds the nominal update interval.

Table 1 lists one core variable per row. Secondary symbols are defined at first use in the surrounding text to avoid an oversized notation table.

The generated ranges ensure a feasible minimum-rate assignment; all returned solutions are checked for unique assignment, legal nodes, finite allocated CPU-cycle rates, selected-node bounds and aggregate capacity.

Table 1. Main Notation

Symbol	Definition
T	Task set

Symbol	Definition
N	Unified device, edge and cloud execution-node set
N_i	Legal execution-node set of task i
$d(i)$	Source device of task i
x_{ii}	Binary assignment of task *i* to node *j*
r_i	Internal normalised CPU coordinate
f_i^{req}	Tentative allocated CPU-cycle-rate request of task i (cycles/s)
f_i	Repaired allocated CPU-cycle rate of task i (cycles/s)
f_i^{min}	Minimum allocatable CPU-cycle rate of node j (cycles/s)
F_i	Aggregate CPU-cycle capacity of node j (cycles/s)
T_i	Tasks assigned to node j after reassignment
n_i	Number of tasks assigned to node j
R_i	CPU-cycle capacity remaining after minimum allocations
N_T	Number of tasks in the decision epoch
N_{act}	Number of active source devices
U_m	Mean base utility of active source device m
Q	Priority-weighted aggregate QoE
J	Jain fairness over active-user mean utilities
$B(X)$	Fixed weighted base objective
$CSR(X)$	Soft threshold constraint-satisfaction ratio
$F_{report}(X)$	Fixed problem objective and cross-algorithm scale
$F_{search}(X,t)$	Iteration-specific internal search fitness
$\lambda(t)$	Dynamic search penalty coefficient
NFE	Number of function evaluations

3.3 QoE and Fairness Metrics

The study uses a model-based base utility u_i rather than a human-subject mean-opinion score. Delay, energy and freshness satisfaction are bounded exponential functions.

Delay satisfaction is

$$S_i^T = \exp\left(-\frac{T_i}{T_i^{max}}\right) \tag{9}$$

Energy satisfaction is

$$S_i^E = \exp\left(-\frac{E_i}{E_i^{bud}}\right) \tag{10}$$

Freshness satisfaction is

$$S_i^A = \exp\left(-\frac{A_i}{A_i^{max}}\right) \tag{11}$$

The priority-neutral base task utility is

$$u_i = 0.45\,S_i^T + 0.30\,S_i^E + 0.25\,S_i^A, 0 < u_i \le 1 \tag{12}$$

Task priority is used only for system QoE aggregation:

$$Q = \frac{\sum_{i \in T} \pi_i u_i}{\sum_{i \in T} \pi_i} \tag{13}$$

For each active source device *m*, let U_m denote the mean base utility over tasks generated by *m*.

This ordering prevents task priority and the number of generated tasks from being counted again inside fairness.

Let D_{act} contain only source devices that generate at least one task in the epoch, and let N_{act} be their count.

$$J = \frac{\left(\sum_{m \in D_{act}} U_m \right)^2}{N_{act} \sum_{m \in D_{act}} U_m^2} \tag{14}$$

$J = 1$ A value of one indicates equal active-user mean base utility. It is QoE-outcome fairness, not equality of allocated CPU-cycle rates.

The parameter ranges are synthetic heterogeneous settings designed to generate capacity-constrained MEC instances; they are not claimed as direct hardware calibration. Their orders of magnitude are consistent with representative MEC system and joint resource-allocation studies [1,3,11,13]. The same versioned ranges are applied to every algorithm and are examined through capacity, SLA and heterogeneity sensitivity analyses.

4 Problem Formulation

P1 combines threshold-normalised device-side energy, delay and AoI with QoE and fairness deficits. These five terms are coupled, because QoE is derived from the first three and AoI contains delay.

The components E^{norm}, D^{norm} and A^{norm} are respectively the task means of device-side energy divided by its budget, delay divided by its maximum and AoI divided by its threshold.

The fixed base objective is

$$B(X) = 0.15\,E^{norm} + 0.15\,D^{norm} + 0.20\,A^{norm} + 0.25(1-Q) + 0.25(1-J) \tag{15}$$

Let N_T be the number of tasks. Soft delay, battery-adjusted energy and AoI checks are summarised by CSR; they do not replace the hard assignment and CPU constraints.

$$CSR(X) = \frac{1}{3N_T} \sum_{i \in T} \left[1(T_i \le T_i^{max}) + 1(E_i \le max\{\beta_i, 0.1\} E_i^{bud}) + 1(A_i \le A_i^{max}) \right] \tag{16}$$

The fixed optimisation problem is therefore defined directly on reporting fitness. The selected-node CPU bound is written as an assignment-weighted bound, so each f_i is constrained only by its chosen node.

In P1, the optimisation variable denotes the decoded physical solution comprising the assignment and allocated-rate decisions, rather than the internal normalised search coordinates. The reporting coefficient is fixed at its prespecified reference value and is not a decision constraint.

$$\begin{aligned} P1: \min_X \; & F_{report}(X) = B(X) + \lambda_{ref}[1 - CSR(X)] \\ s.t. \; & \sum_{j \in N} x_{ij} = 1, x_{ij} \in \{0,1\}, i \in T, \\ & x_{ij} = 0, j \notin N_i, \\ & \sum_{j \in N} x_{ij} f_j^{min} \le f_i \le \sum_{j \in N} x_{ij} F_j, i \in T, \\ & \sum_{i \in T} x_{ij} f_i \le F_j, j \in N. \end{aligned} \tag{17}$$

The soft-violation rate is $v = 1 - CSR$.

The internal normalised coordinates satisfy $0 \le r_i \le 1$ and select only the legal list; they are not additional physical constraints.

The dynamic search fitness used by RDHO is

$$F_{search}(X,t) = B(X) + \lambda(t)\left[1 - CSR(X)\right] \tag{18}$$

$$\lambda(t)=\lambda_0\left(1+\frac{2t}{T_{max}}\right)^{\alpha} \tag{19}$$

Parents and candidates in one greedy selection are evaluated with the same $\lambda(t)$. This prevents comparison under incompatible penalty coefficients.

The search process separately tracks the best fixed-objective candidate seen, denoted by C_{seen}:

$$X^{\star}_{report}=arg\ \min_{X\in C_{seen}} F_{report}(X) \tag{20}$$

Search fitness guides optimisation; reporting fitness is the only cross-algorithm scale in tables and statistics.

Thus P1 is one fixed constrained problem, F_{search} is only a penalty-continuation mechanism for solving it, and F_{report} is the sole ranking scale in tables and statistics.

Problem complexity.

Even without continuous allocation, legal node assignments grow combinatorially. Capacity coupling, exponential utility and indicator-based soft violations make P1 a nonlinear mixed discrete-continuous problem.

The study does not claim a new proof of NP-hardness; it motivates a population search and evaluates empirical solution quality and cost.

A returned incumbent consists of one repaired legal node and one allocated CPU-cycle rate per task.

These characteristics motivate population search over legal categorical assignments and bounded allocated CPU-cycle-rate decisions; no deployment claim is made.

5 RDHO-Based Task-Offloading Strategy

RIME-DBO hybrid optimisation (RDHO) is the focal solver in this study. It adapts RIME-inspired perturbation and DBO-inspired role-conditioned movements to the capacity-feasible MEC encoding, decoder and Algorithm 1 repair path. RDHO-full additionally uses configured seeding and local coordinate refinement; the framework is evaluated both as a complete solver and under controls that isolate its population-update stage. The definitions below reproduce the implemented RDHO rather than requiring the reader to infer its operations from source code.

RIME-inspired and DBO-inspired movements are described as algorithmic components, not as mechanisms presumed to be stronger in advance. Their independent contribution is evaluated under common initialisation, common repair, equal total NFE and common refinement in Section 6.3.

All candidates use the same normalised two-coordinate encoding and are decoded through the common physical repair before evaluation.

RDHO is assessed as a configured capacity-feasible search framework. Controlled experiments, rather than the hybrid label alone, determine which population-update claims are supported.

5.1 Solution Representation and Evaluation Design

Individual Y has N rows $y_i=(u_i,r_i)$, with u_i and r_i in [0,1]. u_i maps by floor(u_i times the number of sorted legal nodes) to a legal node, and r_i decodes to the tentative allocated CPU-cycle rate above. Every generated coordinate is clipped component-wise to [0,1] before Algorithm 1, metric evaluation and hard-feasibility checks. The decoder and Algorithm 1 transform this internal individual into the physical solution used in P1 before evaluation.

With population $P=50$, the first $floor(P/2)$ members are Gaussian: node coordinates are independent $N(0.55,0.25)$ and resource coordinates independent $N(0.60,0.20)$; the remainder are independent $U(0,1)$. RDHO replaces member 0 with a greedy seed: begin every task at (0.5,0.5), visit task indices in ascending order, test the 16 pairs {0.08,0.35,0.62,0.90} times {0.20,0.50,0.80,1.00}, and retain only

a strict fixed-reporting-fitness improvement. Members 1, 2 and 3, when present, are the seed plus independent $N(0, 0.08)$ perturbations, then clipped. These are the three perturbations; no additional trigger is used.

A candidate evaluation returns $B(X)$, $CSR(X)$, dynamic search fitness, fixed reporting fitness, raw metrics, hard-feasibility flags and capacity utilisation.

5.2 RDHO Population-Update Mechanism

At iteration t, $p = t / T_{max}$ and population diversity D_{div} is the mean, over all $2N$ coordinates, of the coordinate-wise population standard deviations. Adaptive producer and scout shares are $\rho_P = clip(0.28 - 0.10\,p + 0.08\,D_{\text{div}}, 0.14, 0.34)$ and $\rho_S = clip(0.08 + 0.08\,D_{\text{div}} + 0.04\,p, 0.08, 0.20)$; $\rho_F = max(0.40, 1 - \rho_P - \rho_S)$. Nominal counts n_P, n_F and n_S are $max(1, floor(P\,\rho_{role}))$ and are computed from the full P; they are not renormalised to $P - n_E$ after $n_E = max(1, floor(0.10\,P))$ elites are reserved. The ranked loop applies deterministic branch precedence: an elite is copied; otherwise $rank < n_P$ gives producer, else $rank < n_P + n_F$ gives follower, else $rank \geq P - n_S$ gives scout, and any remaining member receives $N(0, 0.04(1-p))$ noise. Thus elites truncate nominal roles, and if follower/scout thresholds overlap the earlier follower branch takes precedence; no member is assigned twice. Consequently, ρ_P, ρ_F and ρ_S define nominal role thresholds rather than guaranteed realised population proportions.

For producer current x, search-best x_b and search-worst x_w, draw $\beta \sim N(0,1)$ per coordinate, $\theta \sim U(0, 2\pi)$ per coordinate, $k \sim U(-1, 1)$ per coordinate and $b \sim U(0,1)$. Define the RIME-inspired candidate $v_{RIME} = x_b + \beta \cos(\theta) 2(1-p) 0.28$ and the DBO-inspired candidate $v_{DBO} = x + (1-p)kx + b|x - x_w|$. The producer update is $x' = w\,v_{RIME} + (1-w)\,v_{DBO}$, where $w = 0.5 + 0.3\cos(\pi p)$, decreasing from 0.8 to 0.2.

For a follower, $q = min\left(1, 2\exp\left[-(4p)^2\right]\right)$. With probability q, draw one Bernoulli(0.20) mask per task and replace both coordinates of every masked task by x_b (RIME-inspired puncture). Otherwise draw independent $c_1, c_2 \sim U(0,1)$ per coordinate and use $x' = x_b + c_1 x + c_2(x - 1)$, then clip to [0,1] (bound-aware DBO-inspired foraging).

For scout rank r, set x_L to the member at ranked index floor(0.35r). If its current search fitness exceeds the current best, draw $\theta \sim U(-\pi/4, \pi/4)$ per coordinate and use theft $x' = x_L + \tan(\theta)|x - x_L|$. Otherwise draw standard Cauchy noise z per coordinate and use $x' = x_b + 0.035(1-p)z$. The Cauchy scale is exactly $0.035(1-p)$.

All stated random variables are independent unless their shared task mask is stated. The continuous updates define only a neighbourhood: after clipping, u_i is decoded through that task's legal list and Algorithm 1 always precedes evaluation.

Parents and candidates are evaluated under the same iteration search penalty before strict greedy acceptance. Separately, the reporting incumbent is replaced only by a strict improvement in $F_{report} = B + 1(1 - CSR)$; dynamic search fitness is not reported as a final cross-algorithm result.

Configured RDHO local refinement uses a seeded random permutation of task indices, up to two sweeps, node grid {0.08,0.28,0.48,0.68,0.88}, resource grid {0.10,0.30,0.55,0.80,1.00}, and strict fixed-reporting improvement; it stops after a sweep with no improvement. Controlled common refinement instead uses ascending task order and exactly one 5-by-5 sweep for every method: 25 evaluations per task, or 1,000 NFE for 40 tasks. Experiment A has $50 + 3,750 + 1 = 3,801$ NFE; Experiment B has $50 + 9,181 + 1,000 + 1 = 10,232$ NFE.

For N tasks, M execution nodes, population P, iterations T_{max}, average legal-node count L and refinement cost R, initialisation is $O(PN)$ plus $O(16N)$ greedy tests, role updates are $O(PN)$, and ordinary decoding/evaluation is approximately $O(PNL)$. A typical repair pass is $O(NL)$; retaining this practical cost gives $O(PN + T_{max}PNL + RNL)$ time and $O(PN + NL)$ space. In the worst case repair may

execute up to NM passes, so one candidate can require $O\left(N^2 ML\right)$ repair work and the population-search upper bound includes $O\left(T_{max} P N^2 ML\right)$. This conservative worst case is distinct from the observed zero-failure controlled runs and is not an optimality guarantee.

The complete procedure is summarised in Algorithm 2.

Algorithm 2. Reproducible RDHO-based capacity-feasible joint task-offloading and allocated CPU-cycle-rate strategy

Require:	Scenario, legal-node encoding, Algorithm 1 repair, objective weights, P, T_{max} and a seeded RNG
Ensure:	Hard-feasible fixed-reporting incumbent
1	Generate Gaussian/uniform subpopulations; insert greedy seed and exactly three $N\left(0, 0.08\right)$ seed perturbations when positions 1-3 exist
2	Decode legal nodes, run Algorithm 1, and evaluate B, CSR, current search fitness and fixed reporting fitness
3	for each configured iteration do
4	Compute diversity, adaptive role counts and current search ranking; preserve the top 10% elites
5	Generate the stated producer fusion, follower puncture/foraging, scout theft/Cauchy, or remaining Gaussian candidates; clip to [0,1]
6	Run the shared decoder and Algorithm 1 for every candidate; an unrepaired candidate raises ValueError and aborts the run (no +infinity, parent retention or resampling)
7	Strictly compare parent and candidate under the same current search penalty
8	Strictly update the independent fixed-reporting incumbent
9	end for
10	If configured, apply seeded two-sweep RDHO refinement; controls instead use one common ascending 5-by-5 sweep or none
11	Re-evaluate with λ_{ref}=1 and verify unique legal assignment, allocated-rate bounds and aggregate capacity

Algorithm 1 specifies common deterministic legal-node reassignment and excess-capacity projection used by every solver; Algorithm 2 summarises the implemented RDHO procedure.

6 Performance Evaluation

6.1 Experimental Setup and Reproducibility

The configured V2 end-to-end experiments and additive strictly controlled experiments use the same fixed synthetic scenarios 20260701-20260730, legal-node encoding, allocated-CPU-cycle-rate decoder, deterministic repair, hard-feasibility checks and fixed reporting objective. Controlled artifacts are generated from immutable raw CSV files and audited separately; they do not regenerate or modify results/v2/. The recorded local environment was macOS 26.5.2 on Apple M5 (10 cores, 32 GB memory), Python 3.9.6, NumPy 2.0.2, pandas 2.3.3 and SciPy 1.13.1. Experiment loops use no Python multiprocessing and no explicit thread pinning; all compared methods use the same serial runner configuration. Runtime is a local implementation-cost observation, not a cross-platform benchmark.

The configured end-to-end suite compares RDHO-full, RIME, DBO, TLBO-HHO [21], CWTSSA [36] and Greedy-ED under the same model, utility, repair and fixed reporting objective. RIME, DBO, TLBO-HHO and CWTSSA constants are listed in Table 4 from executable versioned defaults; they remain fixed over every reported scenario, and no algorithm-specific tuning was performed on the reporting scenarios.

The configured end-to-end comparison intentionally retains each solver's configured procedure; NFE is therefore unequal (RDHO-full 10,232, population baselines 7,551 and Greedy-ED 681). It is reported as a complete-solver comparison and cannot isolate the contribution of RDHO's population-update mechanism.

Table 2. Physical system parameters used in the main experiment.

Parameter	Value and interpretation
Devices / edge / cloud	20 / 4 / 2
Tasks	40 heterogeneous tasks
Node CPU capacity	Aggregate CPU-cycle capacity: device 2.2-3.0; edge 18-28; cloud 55-75 Gcycles/s
Minimum allocatable CPU	Minimum allocated CPU-cycle rate: device 0.2; edge 0.8; cloud 1.5 Gcycles/s
Transmit power	0.2-0.8 W
Device-edge rate	8-30 Mbit/s; 10% sparse links, connectivity repaired
Edge-cloud rate	60-150 Mbit/s; 5% sparse links, connectivity repaired
Energy coefficient	$(0.8\text{-}1.4) \times 10^{-27}$ J s^2 cycle^{-3}
Fixed overhead	Edge 0.010 s; cloud 0.055 s
Cloud relay	Fastest legal fixed relay for each selected cloud
Excluded controls	Bandwidth, power, association, routing and queue scheduling

Table 3. Task-generation ranges and sampling probabilities.

Parameter	Compute-intensive	Data-intensive	Real-time	Lightweight
Sampling probability	0.28	0.24	0.28	0.20
Input data (MB)	8-35	30-90	2-15	0.5-5
CPU cycles (Gcycles)	1.8-4.5	0.8-2.2	0.5-1.8	0.1-0.8
Maximum delay (s)	1.5-3.0	2.5-5.0	0.35-1.0	0.8-2.0
AoI threshold (s)	2.0-3.0	3.0-5.0	0.5-1.0	1.0-2.0
Energy budget (J)	2.5-6.0	2.0-5.0	1.0-3.5	0.5-2.0
Battery ratio	0.45-1.0	0.40-0.95	0.55-1.0	0.50-1.0
Priority / interval (s)	0.60-0.90 / 0.50-1.00	0.50-0.80 / 0.80-1.50	0.80-1.00 / 0.10-0.30	0.40-0.70 / 0.30-0.60

Table 4. Reproducibility, RDHO, and baseline parameter settings (fixed across reported scenarios).

Parameter	Value
Configured end-to-end population / iterations	50 / 150

Parameter	Value
Objective weights (energy, delay, AoI, QoE, fairness)	0.15, 0.15, 0.20, 0.25, 0.25
Reporting coefficient	Fixed λ_{ref}=1; fixed-return post-hoc checks at 0.5, 1, 2 only
Paired scenarios	30 (seeds 20260701-20260730)
Experiment A control	Common initial population, decoder and repair; no refinement; 3,801 total NFE
Experiment B control	Common initial population, decoder, repair and deterministic refinement; 10,232 total NFE
Controlled statistics	Two-sided paired Wilcoxon, experiment-local Holm correction, rank-biserial effect size, W/T/L
Compared population methods	RDHO, RIME and DBO
RDHO Gaussian source	Node $N(0.55, 0.25)$; resource $N(0.60, 0.20)$; other half $U(0, 1)$
RDHO greedy seed / perturbations	16 grid pairs in ascending task order; seed plus $N(0, 0.08)$ at positions 1-3
RDHO role / elite constants	producer clip(0.28-0.10p+0.08D_{div},0.14,0.34); scout clip(0.08+0.08D_{div}+0.04p,0.08,0.20); elite 10%
RDHO producer / follower	$w = 0.5 + 0.3\cos(\pi p)$; RIME scale 0.28; puncture $q = min\{1, 2\exp[-(4p)^2]\}$, mask 0.20
RDHO scout / remaining	theft angle $U(-\pi/4, \pi/4)$; Cauchy scale $0.035(1-p)$; remaining $N(0, 0.04(1-p))$
RIME baseline	normal initialisation; $h = 2(1-p)$; exploration scale 0.30; puncture noise 0.035; repository default
DBO baseline	rolling/breeding/foraging/theft 0.20/0.20/0.40/0.20; rolling decay $1-p$; repository default
TLBO-HHO baseline	teaching threshold 0.55; learner-HHO threshold 0.80; teaching factors {1,2}; HHO energy $2(1-p)$; repository default
CWTSSA baseline	producer/scout 0.20/0.10; inertia $0.9 - 0.5p$; t df=3, scale=0.12; Cauchy p=0.20, scale=0.025; repository default
Parameter provenance	Synthetic heterogeneous settings from versioned generator/configuration; not hardware calibrated; no scenario-specific tuning

No algorithm-specific tuning was performed on the reported scenarios. Configurations and exact baseline implementations are versioned with the raw results.

6.2 Configured End-to-End Solver Comparison

Figure 2 reports fixed-reference incumbents during population search. RDHO's final coordinate refinement is not included in its curve, and Greedy-ED is a fixed reference.

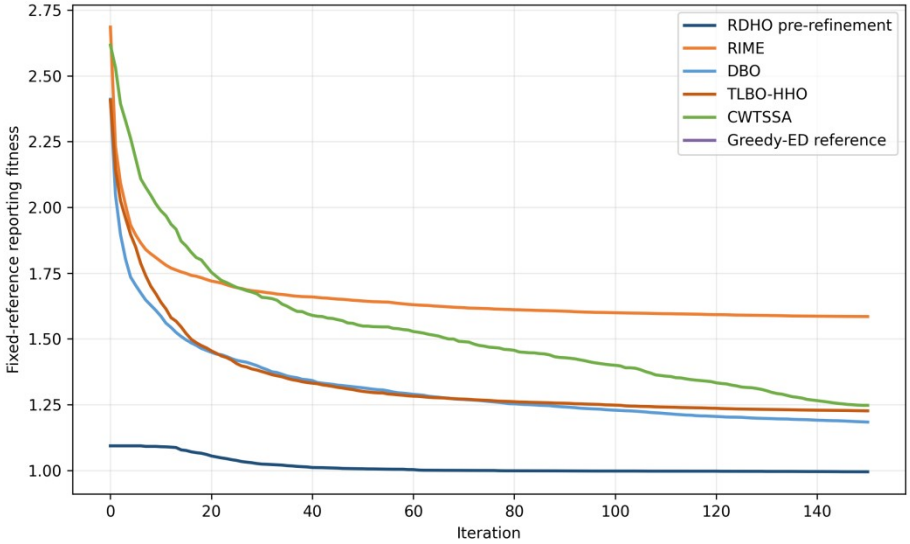

Fig. 2. Fixed-reference reporting-fitness convergence over 150 iterations.

Table 5. Configured end-to-end solver comparison over 30 paired scenarios (mean +/- standard deviation; unequal NFE).

Alg.	Reporting fitness	Device energy (J)	Delay (s)	AoI (s)	QoE	Fairness	Soft CSR	Time (s)	NFE
RDHO -full	0.947 (0.120)	94.58 (24.04)	1.848 (0.218)	2.171 (0.226)	0.447 (0.032)	0.924 (0.020)	0.721 (0.055)	7.831 (0.660)	10232
RIME	1.585 (0.154)	126.37 (30.12)	4.131 (0.795)	4.454 (0.813)	0.355 (0.027)	0.838 (0.040)	0.513 (0.053)	5.966 (0.499)	7551
DBO	1.184 (0.190)	93.19 (31.46)	2.536 (0.479)	2.859 (0.487)	0.415 (0.035)	0.916 (0.026)	0.629 (0.070)	5.820 (0.501)	7551
TLBO-HHO	1.226 (0.201)	60.63 (14.07)	2.766 (0.589)	3.089 (0.593)	0.407 (0.040)	0.910 (0.029)	0.599 (0.072)	5.711 (0.495)	7551
CWTSSA	1.247 (0.151)	62.62 (17.65)	2.775 (0.488)	3.098 (0.496)	0.408 (0.031)	0.914 (0.024)	0.588 (0.057)	5.678 (0.486)	7551
Greedy -ED	1.093 (0.170)	99.75 (22.47)	2.225 (0.538)	2.548 (0.550)	0.428 (0.034)	0.916 (0.032)	0.648 (0.062)	0.507 (0.044)	681

RDHO-full has the lowest configured mean reporting fitness (0.947 +/- 0.120) and every returned solution is hard feasible. Because RDHO-full uses 10,232 NFE while the population baselines use 7,551, this result supports the configured end-to-end solver rather than isolated superiority of its hybrid population update.

Metric-specific leaders differ: TLBO-HHO has the lowest device-side energy, RDHO-full has the lowest delay and AoI and the highest QoE, active-user fairness and CSR, while Greedy-ED is fastest. RDHO-full also consumes more NFE.

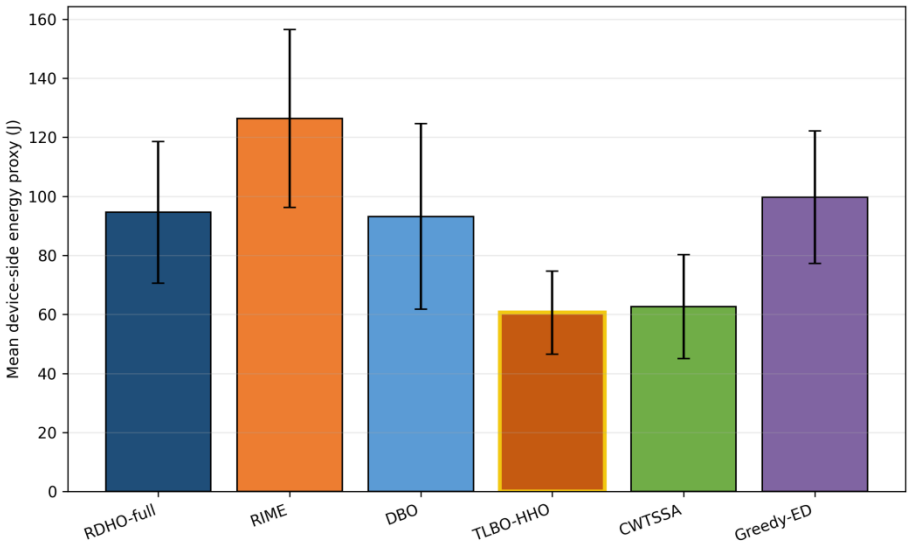

Fig. 3. Mean device-side energy over 30 paired runs.

TLBO-HHO is the energy leader; RDHO optimises energy as one component of the coupled reporting preference.

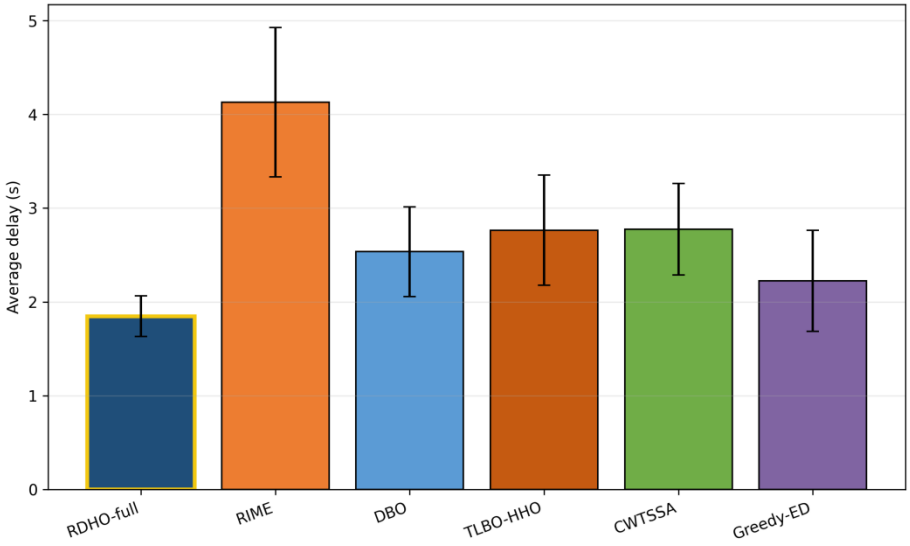

Fig. 4. Mean processing delay over 30 paired runs.

RDHO-full has the lowest mean delay under the configured end-to-end procedures.

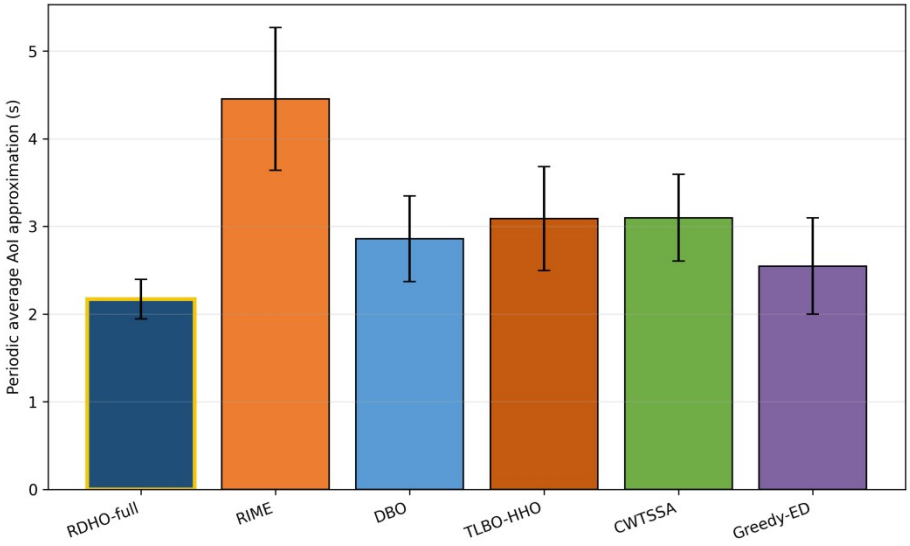

Fig. 5. Periodic no-backlog average-AoI approximation over 30 paired runs.

RDHO-full has the lowest mean AoI approximation; the close ordering with delay follows Eq. (8).

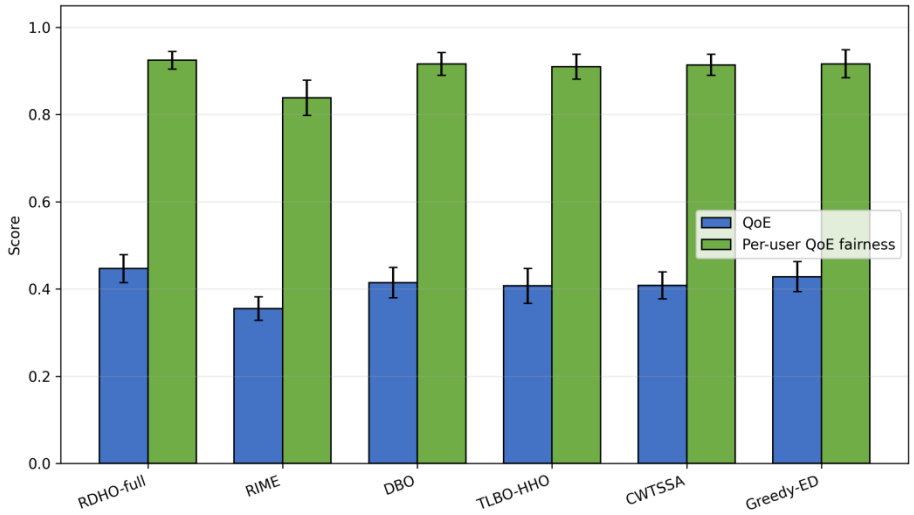

Fig. 6. Priority-weighted aggregate QoE and active-user base-utility fairness.

RDHO-full has the highest mean QoE, whereas fairness differences are small and do not imply equality of CPU allocations.

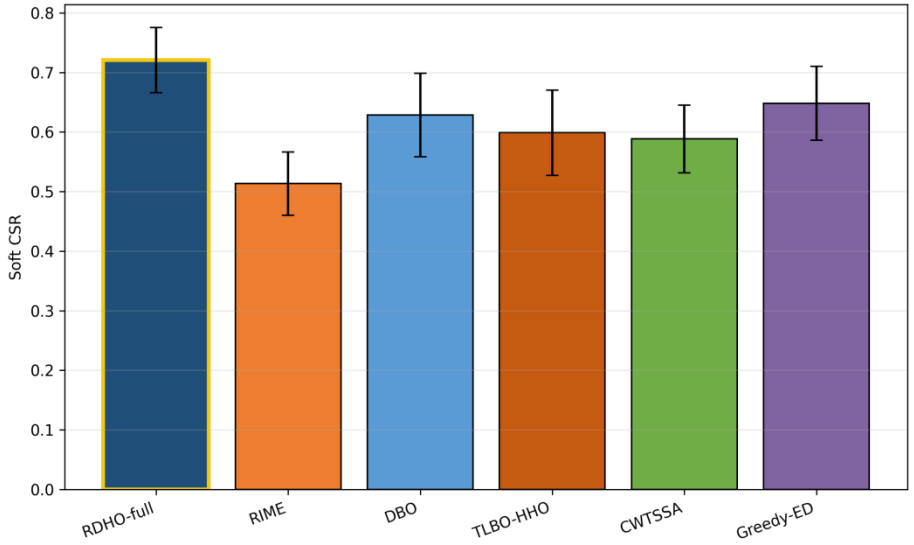

Fig. 7. Soft constraint-satisfaction ratio (CSR).

RDHO-full reaches mean CSR 0.721. CSR remains a binary threshold diagnostic; it does not imply that every task satisfies every soft threshold.

Configured-comparison statistics.

Two-sided paired Wilcoxon tests use the 30 matched reporting-fitness values. Holm adjustment controls the family-wise error rate; median paired difference, signed rank-biserial correlation and wins/ties/losses report magnitude and consistency. In this subsection, they test complete configured procedures with unequal NFE.

Table 6. Paired Wilcoxon tests for configured end-to-end solvers (unequal NFE).

Comparison	W	Two-sided p	Holm p	Median diff.	Signed r_{rb}	W/T/L
RDHO-full vs RIME	0	1.86e-09	9.31e-09	-0.6454	-1.000	30/0/0
RDHO-full vs DBO	0	1.86e-09	9.31e-09	-0.2030	-1.000	30/0/0
RDHO-full vs TLBO-HHO	0	1.86e-09	9.31e-09	-0.2581	-1.000	30/0/0

Comparison	W	Two-sided p	Holm p	Median diff.	Signed r_{rb}	W/T/L
RDHO-full vs CWTSSA	0	1.86e-09	9.31e-09	-0.2979	-1.000	30/0/0
RDHO-full vs Greedy-ED	0	1.86e-09	9.31e-09	-0.1029	-1.000	30/0/0

RDHO-full is lower in all 30 pairs against each configured main baseline. These tests concern complete procedures with their configured initialisation, population search, incumbent tracking and refinement; they do not prove an isolated hybrid-operator advantage.

6.3 Strictly Controlled RDHO Population Evidence

Two strictly controlled paired experiments answer the attribution question directly. Every method receives one identical 50-by-40-by-2 initial population per scenario, follows the common legal-node decoder and Algorithm 1 repair, reports the same fixed reporting fitness and is constrained by an exact total NFE budget. All 90 returns in each experiment are hard feasible. Table 7 reports reporting fitness, base objective B, soft CSR, QoE, fairness, NFE and runtime. The primary endpoint is lower fixed reporting fitness; paired differences $\Delta F = F_{RDHO} - F_{parent}$ are negative when RDHO is lower. Experiment A (no refinement, 3,801 NFE) gives DBO a clear advantage: RDHO versus DBO median ΔF=+0.178910, Holm p=3.725e-09 and W/T/L=0/0/30. At the prespecified λ_{ref}=1, Experiment B (common refinement, 10,232 NFE) gives DBO a smaller but statistically significant advantage: means 0.9732 versus 0.9664, difference 0.006801; median ΔF=+0.008226, Holm p=0.026229 and W/T/L =10/0/20. RDHO remains lower than RIME in both controls.

Table 7. Strictly controlled fixed-return evidence over 30 paired scenarios: reporting fitness and components, QoE, fairness, NFE and runtime.

Experiment	Method	F_{report} / B / CSR (mean)	QoE	Fairness	NFE	Runtime (s)
A: population stage	RDHO	1.4188 / 0.9727 / 0.5539	0.3937	0.8963	3801	3.149
A: population stage	RIME	1.6834 / 1.1701 / 0.4867	0.3456	0.8476	3801	3.154
A: population stage	DBO	1.2409 / 0.8504 / 0.6094	0.4096	0.9120	3801	3.053
B: common pipeline	RDHO	0.9732 / 0.6779 / 0.7047	0.4460	0.9273	10232	9.054
B: common pipeline	RIME	0.9836 / 0.6758 / 0.6922	0.4454	0.9267	10232	9.169
B: common pipeline	DBO	0.9664 / 0.6745 / 0.7081	0.4460	0.9261	10232	8.930
Paired tests	**RDHO comparison**	**Median ΔF [95% CI]**	**Holm p**	**r_{rb}**	**W/T/L**	
A	RDHO vs RIME	-0.278300 [-0.312822, -0.198123]	1.304e-08	-0.983	29/0/1	
A	RDHO vs DBO	+0.178910 [+0.112584, +0.225303]	3.725e-09	+1.000	0/0/30	
B	RDHO vs RIME	-0.012281 [-0.024215, -0.001839]	0.019863	-0.531	23/0/7	

Experiment	Method	F_{report} / B / CSR (mean)	QoE	Fairness	NFE	Runtime (s)
B	RDHO vs DBO	+0.008226 [+0.000775, +0.014349]	0.026229	+0.462	10/0/20	

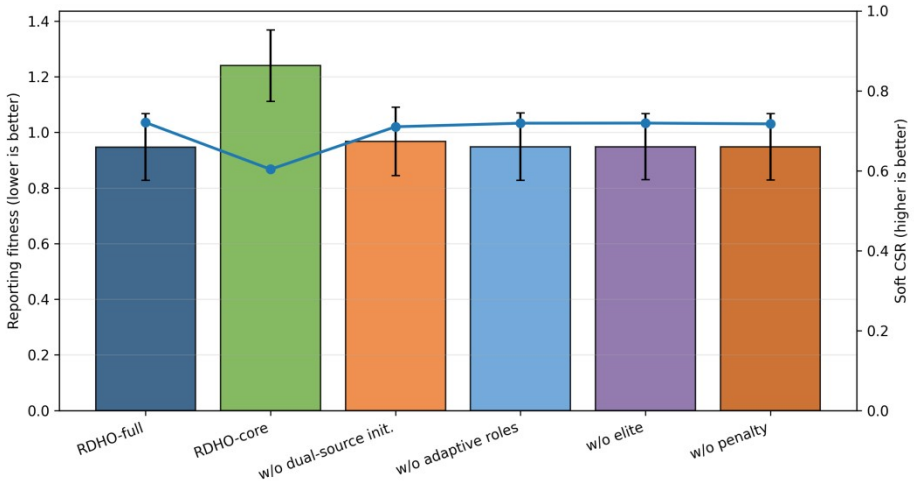

Fig. 8. Reporting fitness and soft CSR for configured RDHO variants.

The configured ablation remains descriptive for the end-to-end RDHO workflow. Coordinate refinement changes configured mean fitness from 1.240 to 0.947, so it is one component of the complete-pipeline result; population-update attribution is reported in Table 7 above, not inferred from this configured ablation.

6.4 Component, Refinement, Scalability, and Sensitivity Analyses

Configured component/refinement analysis begins here. It is descriptive for the complete RDHO workflow, whereas Table 7 provides the formal common-initialisation, common-decoder, common-repair, exact-NFE and common-refinement attribution evidence. Additional configured component-comparison results are provided in the repository supplement to avoid confusion with the strict controls.

Table 8. RDHO scalability under 20-100 tasks.

Tasks	Reporting fitness	Soft CSR	Time (s)	NFE
20	0.897 (0.124)	0.745 (0.059)	3.430 (0.056)	8892
40	0.977 (0.116)	0.708 (0.055)	6.641 (0.119)	10232
60	0.998 (0.106)	0.698 (0.050)	10.370 (0.128)	11572
80	1.059 (0.100)	0.665 (0.041)	14.833 (0.226)	12912
100	1.135 (0.070)	0.633 (0.030)	19.772 (0.220)	14252

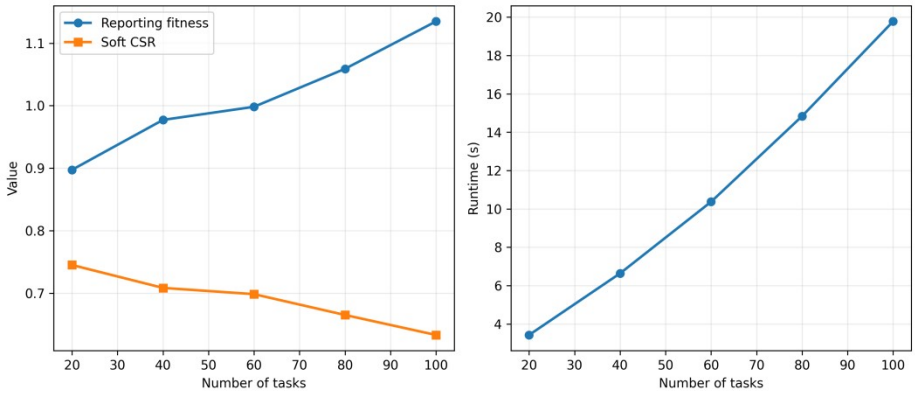

Fig. 9. Reporting fitness, soft CSR and runtime versus task count.

The scalability suite uses a separate scenario set for each task count and is independent of the 30-paired-scenario configured main comparison; therefore, its 40-task row is not expected to reproduce the main-comparison mean of 0.947. Scalability and configured sensitivity describe the RDHO framework outside the strict attribution controls and do not override Table 7. Dynamic-search-penalty sensitivity changes only search guidance; fixed reporting fitness remains B+1(1-CSR).

Component and refinement interpretation.

The controlled results indicate that the implemented RDHO population mechanism improves substantially over RIME but not DBO in the fixed V2 scenarios. The contribution is therefore interpreted at the integrated-framework level, not as universal population-update superiority. Common refinement reduces the absolute DBO gap but does not reverse it at λ_{ref}=1.

The final sensitivity ranges are generated directly from V2 CSV files; no value is manually entered into a paper table.

Weight changes expose engineering preference rather than a universal optimum. Task-utility coefficient sensitivity checks the internal QoE construction, while CPU-capacity, SLA and server-heterogeneity scaling test load, threshold strictness and heterogeneous legal server choices.

Settings S1-S7 denote the seven prespecified objective-weight vectors used in this sensitivity analysis; their exact values are provided in the versioned repository configuration and accompanying summary files.

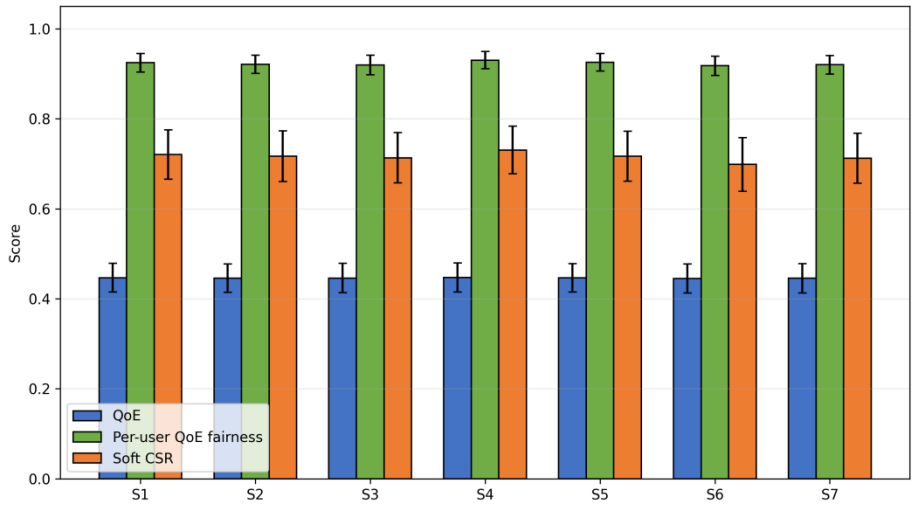

Fig. 10. Objective-weight sensitivity of QoE, active-user fairness and soft CSR.

The penalty heatmaps compare returned solutions under the common fixed reporting coefficient λ_{ref} =1; they do not alter the reporting definition.

Table 9. Dynamic-penalty sensitivity under fixed reporting fitness.

Initial penalty	Growth exponent	Reporting fitness	QoE	Fairness	Soft CSR	Time (s)
0.5	1.0	0.947 (0.119)	0.447 (0.032)	0.924 (0.020)	0.719 (0.057)	6.661 (0.113)
0.5	2.0	0.948 (0.118)	0.447 (0.032)	0.924 (0.020)	0.719 (0.055)	6.606 (0.106)
0.5	3.0	0.948 (0.119)	0.447 (0.033)	0.925 (0.020)	0.720 (0.055)	6.519 (0.109)
1.0	1.0	0.949 (0.119)	0.447 (0.033)	0.924 (0.020)	0.719 (0.054)	6.521 (0.084)
1.0	2.0	0.947 (0.120)	0.447 (0.032)	0.924 (0.020)	0.721 (0.055)	6.514 (0.113)
1.0	3.0	0.948 (0.121)	0.446 (0.033)	0.924 (0.020)	0.721 (0.054)	6.536 (0.113)
2.0	1.0	0.948 (0.120)	0.447 (0.032)	0.925 (0.020)	0.721 (0.055)	6.532 (0.106)
2.0	2.0	0.948 (0.119)	0.447 (0.032)	0.925 (0.020)	0.721 (0.053)	6.534 (0.118)
2.0	3.0	0.950 (0.121)	0.446 (0.033)	0.925 (0.020)	0.722 (0.054)	6.520 (0.109)

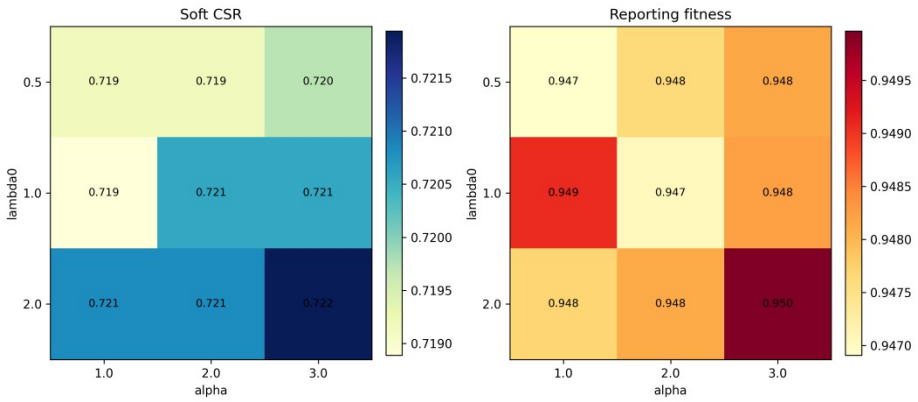

Fig. 11. Dynamic-penalty sensitivity heatmaps for soft CSR and reporting fitness.

The nine penalty schedules are reported without post-hoc selection. Full objective-weight, task-utility, CPU-capacity, SLA and server-heterogeneity tables remain in the repository supplement and extend the sensitivity boundary without post-hoc tuning claims.

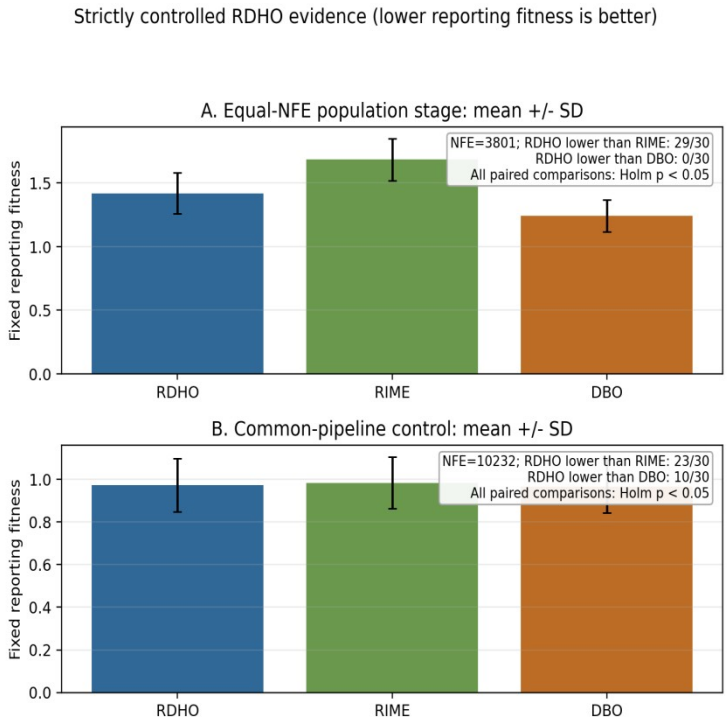

Fig. 12. Strictly controlled paired reporting-fitness evidence. Panels A and B use 3,801 and 10,232 total NFE, respectively; negative RDHO-minus-parent differences favour RDHO.

Figure 12 visualises both controls rather than hiding the adverse comparison: RDHO is lower than RIME in 29/30 population-stage pairs and 23/30 common-pipeline pairs, but lower than DBO in 0/30 and 10/30 pairs. For fixed returned solutions, post-hoc rescoring $F_\lambda = B + \lambda_{ref}(1 - CSR)$ at λ_{ref} in 0.5, 1 and 2 does not re-optimise or select another incumbent. DBO remains significantly lower than RDHO in Experiment A at all three values (Holm p <= 1.118e-08); in Experiment B it remains significant at 0.5 (Holm p=0.027326) and 1 (Holm p=0.026229), but not at 2 (Holm p=0.104840). Thus the small Experiment-B DBO advantage is penalty-definition-sensitive for fixed returns, while Experiment-A is robust. The configured RDHO-full result demonstrates the complete framework; isolated population-update evidence does not support superiority over DBO.

7 Conclusion

This work formulated a capacity-feasible MEC joint task-offloading and allocated CPU-cycle-rate model and developed RDHO as its focal integrated solver framework. Each task selects one legal source-local, reachable-edge or reachable-cloud node, and deterministic repair enforces allocated-rate bounds and aggregate CPU-cycle capacity.

In the configured end-to-end comparison, RDHO-full has the lowest mean fixed reporting fitness (0.947) with hard feasibility in every returned solution, but this comparison has unequal NFE and evaluates complete solver configurations. The strict controls disclose the narrower result: at 3,801 NFE without refinement, RDHO is significantly lower than RIME but significantly higher than DBO. At the prespecified reporting coefficient λ_{ref}=1, RDHO is again significantly lower than RIME but slightly and significantly higher than DBO under the 10,232-NFE common-refinement control; this smaller DBO advantage is not statistically significant in the fixed-return post-hoc rescoring at λ_{ref}=2. Accordingly, the evidence supports the RDHO-based capacity-feasible framework and its controlled evaluation methodology, not independent or universal superiority of the RIME-DBO population update.

Limitations include synthetic offline tasks, fixed-rate communication, deterministic cloud relay selection, device-side rather than infrastructure energy, a periodic no-backlog AoI approximation, coupled objective terms, engineering utility coefficients and the fixed set of implemented algorithms and 30 scenarios. Energy, delay and AoI also enter QoE, while delay enters AoI and all three reappear in CSR; the current study does not isolate the incremental contribution of each nested objective layer.

The current implementation aborts an optimiser run if deterministic candidate repair fails, although no such failure occurred in the reported controlled experiments. Future work can investigate objective-layer ablation, adaptive operator selection or DBO-dominant RDHO variants, queue-aware arrivals, communication-resource optimisation, infrastructure energy, calibrated QoE and physical testbeds without claiming that the present hybrid operator has already surpassed DBO.

CRediT authorship contribution statement

Haoru Su: Conceptualization, methodology, supervision, Writing - review and editing. Jie Bai: Methodology, validation, formal analysis, Writing - review and editing. Jiangyi Zhou: Software, data curation, visualization, Writing - original draft, Writing - review and editing. All authors reviewed and approved the final manuscript.

Conflicts of Interest Declaration

All authors declare that they have no conflicts of interest.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Declaration of Generative AI and AI-Assisted Technologies in Manuscript Preparation

During the preparation of this work, the authors used ChatGPT and Codex for language editing, code assistance, experimental scripting, statistical-analysis support, manuscript restructuring and formatting checks. All assisted code, analyses, numerical outputs, figures and manuscript text were reviewed, tested and verified by the authors, who take full responsibility for the content of the publication.

Data Availability

The project repository containing the implementation, versioned configurations, synthetic scenario generator, raw and summary result files, statistical-analysis outputs, manuscript tables, figures, and reproduction instructions is publicly available at https://github.com/Ryan-Yii/mec-rdho-offloading. The v2.0.0 GitHub release provides an immutable snapshot of experiment baseline 0264b6d, the verification fixes, the aligned manuscript, and the reproducibility materials. No proprietary, confidential, human-subject, or third-party restricted data were used; all study data were generated synthetically.

References

[1] Mao, Y., You, C., Zhang, J., Huang, K. and Letaief, K.B. (2017) 'A survey on mobile edge computing: the communication perspective', IEEE Communications Surveys & Tutorials, Vol. 19, No. 4, pp.2322-2358, doi: 10.1109/COMST.2017.2745201.

[2] Taleb, T., Samdanis, K., Mada, B., Flinck, H., Dutta, S. and Sabella, D. (2017) 'On multi-access edge computing: a survey of the emerging 5G network edge cloud architecture and orchestration', IEEE Communications Surveys & Tutorials, Vol. 19, No. 3, pp.1657-1681, doi: 10.1109/COMST.2017.2705720.

[3] Sardellitti, S., Scutari, G. and Barbarossa, S. (2015) 'Joint optimization of radio and computational resources for multicell mobile-edge computing', IEEE Transactions on Signal and Information Processing over Networks, Vol. 1, No. 2, pp.89-103, doi: 10.1109/TSIPN.2015.2448520.

[4] Tang, Z., Sun, Z., Yang, N. and Zhou, X. (2023) 'Age of information of multi-user mobile edge computing systems', IEEE Open Journal of the Communications Society, Vol. 4, pp.1600-1614, doi: 10.1109/OJCOMS.2023.3294942.

[5] Skorin-Kapov, L., Varela, M., Hossfeld, T. and Chen, K.-T. (2018) 'A survey of emerging concepts and challenges for QoE management of multimedia services', ACM Transactions on Multimedia Computing, Communications, and Applications, Vol. 14, No. 2s, pp.1-29, doi: 10.1145/3176648.

[6] Luo, J., Deng, X., Zhang, H. and Qi, H. (2019) 'QoE-driven computation offloading for edge computing', Journal of Systems Architecture, Vol. 97, pp.34-39, doi: 10.1016/j.sysarc.2019.01.019.

[7] Zhou, J. and Zhang, X. (2021) 'Fairness-aware task offloading and resource allocation in cooperative mobile-edge computing', IEEE Internet of Things Journal, Vol. 9, No. 5, pp.3812-3824, doi: 10.1109/JIOT.2021.3100253.

[8] Dong, S., Tang, J., Abbas, K., Hou, R., Kamruzzaman, J., Rutkowski, L. and Buyya, R. (2024) 'Task offloading strategies for mobile edge computing: a survey', Computer Networks, Vol. 254, Article 110791, doi: 10.1016/j.comnet.2024.110791.

[9] Wang, D., Bin Abu Bakar, K., Isyaku, B., Eisa, T.A.E. and Abdelmaboud, A. (2024) 'A comprehensive review on Internet of Things task offloading in multi-access edge computing', Heliyon, Vol. 10, No. 9, Article e29916, doi: 10.1016/j.heliyon.2024.e29916.

[10] Wu, H., Lu, Y., Ma, H., Xing, L., Deng, K. and Lu, X. (2025) 'A survey on task type-based computation offloading in mobile edge networks', Ad Hoc Networks, Vol. 169, Article 103754, doi: 10.1016/j.adhoc.2025.103754.

[11] Zhang, K., Mao, Y., Leng, S., Zhao, Q., Li, L., Peng, X., Pan, L., Maharjan, S. and Zhang, Y. (2016) 'Energy-efficient offloading for mobile edge computing in 5G heterogeneous networks', IEEE Access, Vol. 4, pp.5896-5907, doi: 10.1109/ACCESS.2016.2597169.
[12] Almuseelem, W. (2025) 'Deep reinforcement learning-enabled computation offloading: a novel framework to energy optimization and security-aware in vehicular edge-cloud computing networks', Sensors, Vol. 25, No. 7, Article 2039, doi: 10.3390/s25072039.
[13] An, X., Li, Y., Chen, Y. and Li, T. (2024) 'Joint task offloading and resource allocation for multi-user collaborative mobile edge computing', Computer Networks, Vol. 250, Article 110604, doi: 10.1016/j.comnet.2024.110604.
[14] Chen, T., Tan, F., Ai, J., Xiong, X., Wu, C. and Ren, X. (2024) 'Joint optimization of task offloading and service placement for digital twin empowered mobile edge computing', Proceedings of the 2024 3rd International Conference on Networks, Communications and Information Technology, pp.132-137, doi: 10.1145/3672121.3672145.
[15] Huang, L., Bi, S. and Zhang, Y.-J.A. (2019) 'Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks', IEEE Transactions on Mobile Computing, Vol. 19, No. 11, pp.2581-2593, doi: 10.1109/TMC.2019.2928811.
[16] Liu, L., Feng, J., Mu, X., Pei, Q., Lan, D. and Xiao, M. (2023) 'Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing', IEEE Transactions on Intelligent Transportation Systems, Vol. 24, No. 12, pp.15513-15526, doi: 10.1109/TITS.2023.3249745.
[17] Zang, L., Zhang, X. and Guo, B. (2022) 'Federated deep reinforcement learning for online task offloading and resource allocation in WPC-MEC networks', IEEE Access, Vol. 10, pp.9856-9867, doi: 10.1109/ACCESS.2022.3144415.
[18] Abd-Elmagid, M.A., Pappas, N. and Dhillon, H.S. (2019) 'On the role of age of information in the Internet of Things', IEEE Communications Magazine, Vol. 57, No. 12, pp.72-77, doi: 10.1109/MCOM.001.1900041.
[19] Jin, L., Tang, M., Pan, J., Zhang, M. and Wang, H. (2024) 'Asynchronous fractional multi-agent deep reinforcement learning for age-minimal mobile edge computing', arXiv:2409.16832.
[20] Pei, Y., Zhao, Y. and Hou, F. (2024) 'Minimizing age of information in UAV-assisted edge computing system with multiple transmission modes', Tsinghua Science and Technology, Vol. 30, No. 3, pp.1060-1078, doi: 10.26599/TST.2024.9010046.
[21] Bai, J., Su, H., Bi, J., Zhang, L., Yuan, X. and Liu, X. (2025) 'TLBO-HHO AoI-aware task offloading for mobile edge computing', Proceedings of the 2025 IEEE International Conference on Smart Internet of Things (SmartIoT), pp.88-95, doi: 10.1109/SmartIoT66867.2025.00021.
[22] Fiedler, M., Hossfeld, T. and Tran-Gia, P. (2010) 'A generic quantitative relationship between quality of experience and quality of service', IEEE Network, Vol. 24, No. 2, pp.36-41, doi: 10.1109/MNET.2010.5430142.
[23] Jain, R.K., Chiu, D.M. and Hawe, W.R. (1984) 'A quantitative measure of fairness and discrimination for resource allocation in shared computer systems', Digital Equipment Corporation, Hudson, MA, Technical Report DEC-TR-301.
[24] Kelly, F.P., Maulloo, A.K. and Tan, D.K.H. (1998) 'Rate control for communication networks: shadow prices, proportional fairness and stability', Journal of the Operational Research Society, Vol. 49, No. 3, pp.237-252, doi: 10.1038/sj.jors.2600523.
[25] Le Callet, P., Moller, S. and Perkis, A. (eds) (2013) 'Qualinet white paper on definitions of quality of experience', European Network on Quality of Experience in Multimedia Systems and Services.
[26] Ling, C., Zhang, W., He, H., Yadav, R., Wang, J. and Wang, D. (2024) 'QoS and fairness oriented dynamic computation offloading in the Internet of Vehicles based on estimated time of arrival', IEEE Transactions on Vehicular Technology, Vol. 73, No. 7, pp.10554-10571, doi: 10.1109/TVT.2024.3364669.
[27] Chen, H., Heidari, A.A., Chen, H., Wang, M., Pan, Z. and Gandomi, A.H. (2020) 'Multi-population differential evolution-assisted Harris hawks optimization: framework and case studies', Future Generation Computer Systems, Vol. 111, pp.175-198, doi: 10.1016/j.future.2020.04.008.
[28] Deb, K., Pratap, A., Agarwal, S. and Meyarivan, T. (2002) 'A fast and elitist multiobjective genetic algorithm: NSGA-II', IEEE Transactions on Evolutionary Computation, Vol. 6, No. 2, pp.182-197, doi: 10.1109/4235.996017.
[29] Deb, K., Sindhya, K. and Hakanen, J. (2016) 'Multi-objective optimization', in Decision Sciences: Theory and Practice, CRC Press, Boca Raton, FL, pp.161-200.
[30] Heidari, A.A., Mirjalili, S., Faris, H., Aljarah, I., Mafarja, M. and Chen, H. (2019) 'Harris hawks optimization: algorithm and applications', Future Generation Computer Systems, Vol. 97, pp.849-872, doi: 10.1016/j.future.2019.02.028.
[31] Li, Z. and Zhu, Q. (2020) 'Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing', Information, Vol. 11, No. 2, Article 83, doi: 10.3390/info11020083.
[32] Rao, R.V., Savsani, V.J. and Vakharia, D.P. (2011) 'Teaching-learning-based optimization: a novel method for constrained mechanical design optimization problems', Computer-Aided Design, Vol. 43, No. 3, pp.303-315, doi: 10.1016/j.cad.2010.12.015.
[33] Zhang, Q. and Li, H. (2007) 'MOEA/D: a multiobjective evolutionary algorithm based on decomposition', IEEE Transactions on Evolutionary Computation, Vol. 11, No. 6, pp.712-731, doi: 10.1109/TEVC.2007.892759.
[34] Su, H., Zhao, D., Heidari, A.A., Liu, L., Zhang, X., Mafarja, M. and Chen, H. (2023) 'RIME: a physics-based optimization', Neurocomputing, Vol. 532, pp.183-214, doi: 10.1016/j.neucom.2023.02.010.
[35] Xue, J. and Shen, B. (2023) 'Dung beetle optimizer: a new meta-heuristic algorithm for global optimization', The Journal of Supercomputing, Vol. 79, No. 7, pp.7305-7336, doi: 10.1007/s11227-022-04959-6.
[36] Yang, X., Liu, J., Liu, Y., Xu, P., Yu, L., Zhu, L., Chen, H. and Deng, W. (2021) 'A novel adaptive sparrow search algorithm based on chaotic mapping and t-distribution mutation', Applied Sciences, Vol. 11, No. 23, Article 11192, doi: 10.3390/app112311192.
[37] Ma, C., Zhu, J., Liu, M., Zhao, H., Liu, N. and Zou, X. (2021) 'Parking edge computing: parked-vehicle-assisted task offloading for urban VANETs', IEEE Internet of Things Journal, Vol. 8, No. 11, pp.9344-9358, doi: 10.1109/JIOT.2021.3056396.
[38] Wu, H., Wolter, K., Jiao, P., Deng, Y., Zhao, Y. and Xu, M. (2021) 'EEDTO: an energy-efficient dynamic task offloading algorithm for blockchain-enabled IoT-edge-cloud orchestrated computing', IEEE Internet of Things Journal, Vol. 8, No. 4, pp.2163-2176, doi: 10.1109/JIOT.2020.3033521.
[39] Feng, J., Yu, F.R., Pei, Q., Chu, X., Du, J. and Zhu, L. (2020) 'Cooperative computation offloading and resource allocation for blockchain-enabled mobile-edge computing: a deep

reinforcement learning approach', IEEE Internet of Things Journal, Vol. 7, No. 7, pp.6214-6228, doi: 10.1109/JIOT.2019.2961707.
[40] Yuan, X., Tian, H., Wang, H., Su, H., Liu, J. and Taherkordi, A. (2020) 'Edge-enabled WBANs for efficient QoS provisioning healthcare monitoring: a two-stage potential game-based computation offloading strategy', IEEE Access, Vol. 8, pp.92718-92730, doi: 10.1109/ACCESS.2020.2992639.